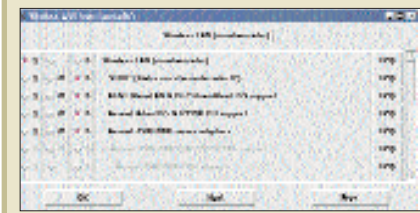


Kernel Secrets

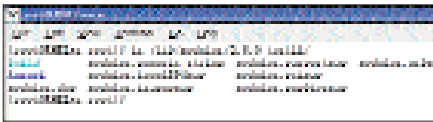
About 'make'

All throughout this workshop, and in many of the previous workshops, we keep bumping into this command called **'make'**, every time we have to compile and install any application. What exactly is make, and why do we keep appending it with words such as 'install', 'clean', etc.? The secret of all this lies in a single 'Makefile' that resides in the directory in which you are compiling the software. Typically, this is the main directory that is created once you untar a 'xxx.tar.gz' or 'xxx.tar.bz2' file using the **tar -zxvf** or **tar -jxvf** commands. This Makefile has all the directions that the make utility needs to compile and install the application. But all these instructions are not just executed in one single go. Instead, various groups of commands, called targets, are clubbed together in different sections of the Makefile. The word that is appended to make utility is, in fact, the name of the target group, and can be found in the Makefile.



/lib/modules/version-number directory, where 'version-number' should be replaced by the version number of the kernel being compiled (2.6.0-test11 in this case). This is done using the **make modules_install** command.

Check if all the modules were copied



All the new modules are installed in the **/lib/modules/2.6.0-test11** directory to the destination by typing **ls /lib/modules/version-number**



After configuring and recompiling the kernel, you need to copy it to the **/boot** directory along with your existing, stable kernel, and then let the boot loader know of the existence of this new kernel.

Copy the bzImage file to the **/boot** directory using the **cp /usr/src/linux-2.6.0-test11/arch/i386/boot/bzImage /boot/bzImage-2.6.0-test11** command.

Linux kernel development tree:

The kernel typically has two releases out at a time. One is the 'Stable' release—which is currently the 2.6.X series—and one is the 'Development' release, which is currently 2.5.X. Stable kernels are well tested and have passed through a great deal of quality assurance testing and bug fixes. Development kernels are experimental with a lot of new, untested code. These are not recommended for production machines. Most Linux distributions ship the latest stable kernel. The stable kernel is always an even number, say 2.2.XX or 2.4.XX. The development kernel is always an odd number, like 2.3.XX, or 2.5.XX.

Cross-compiling:

Linux is an operating system that has been ported to a multitude of platforms. Today, Linux runs on almost any architecture currently available. This has been of immense help to software developers, since the source code for their applications can be compiled on all these platforms with minimal trouble. This is possible because almost all the development tools have also been ported to the respective architectures. In particular, there's a feature in 'gcc' that lets developers cross-compile source code to

generate executable files that can be run on a platform other than the one on which it was compiled. What this means is that a simple application, let's say an mp3 player, that was cross-compiled using gcc on an Intel x86 pc to run on a PowerPC processor, can be just copied to the PPC machine and run as though it were compiled on it.

This cross-compiling is not just restricted to Linux applications, the Linux kernel can itself be cross-compiled to run on another platform. So using the same source code and the same compiler, you can create a new kernel image than can run on your Mac running Yellow Dog Linux.

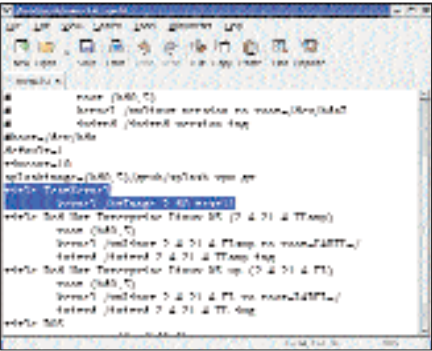
You don't necessarily have to compile the kernel just to install a new version of the kernel. You could use recompilation to switch on certain features in your existing kernel that were not enabled by the vendor—Red Hat, SuSE, etc. For example, if you just bought a Aironet wireless adapter, and want to configure it to run on Linux, chances are you won't have to scout the net to get a driver module; it will be right there in the kernel source tree. You can configure the driver to be either built into your kernel, or you can build it as a kernel module during the 'make xconfig' stage.

In LILO, you just have to update the **/etc/lilo.conf** file and type in **lilo** at the command prompt. If you are using GRUB as the bootloader, you will have to follow a slightly different approach. Use the 'grubby' command to add the newly created kernel to the GRUB boot menu as follows:

grubby --add-kernel=/boot/bzImage-2.6.0-test11 --title=TestKernel

This command will add a new entry to the GRUB boot menu file, **/boot/grub/menu.lst**. Make sure that the title parameter is passed to grubby, since this is the option that will be added to the boot menu that appears during bootup. Open the file in a text editor and check if the entry was added correctly.

That's it, we're almost done. Just reboot your machine, and select the TestKernel entry in the boot menu to boot into your new kernel. If everything went fine, you should not notice any difference between your previous GUI and the present one. To verify that the newer kernel is being used, use the



The 'grubby' command will add an entry for the new kernel in the menu.lst file

uname -r command.

But this is just the beginning; once you start using the new kernel, you will be faced with a number of problems and will have to work on these. But the added advantages you get out of using one of these bleeding-edge kernels outweigh the trouble involved in installing a new one.

PRAVEEN KURUP
praveen_kurup@thinkdigit.com